Question 1

a) Snooping requires messages to be sent across a shared bus. If a shared bus doesn't exist, directory-based coherence is a good implementation alternative. Scalability wise, directory is better than snooping because contention in the bus can be significant if there are too many snoopers.

b) On each cache block we need: 1 dirty bit, 8 bits to store the owner, 1 bit to indicate the owner pointer is valid, and the directory for 256 caches of 1 bit each.

i. (256+10) / (256 + 266) = 0.51. **// Correct.**
ii. (256+10) /256 = 1.039. // Also accepted for HW.

c) On each cache block we need: 10 bits from part b, 8 pointers, and 1 invalid bit to tell whether more than 8 caches have the block. Each pointer has 8 bits to point to a cache, and 1 invalid bit to tell whether or not the pointer is valid.

10 + 1 + 8*(8+1) = 11 + 72 = 83,
i. 83 / (83 + 256) = 0.245. **// Correct.**
ii. 83/256 = 0.32. // Also accepted for HW.

d) We need to come up with a solution for the case when the number of sharers exceed 8, otherwise the system will enter an invalid state. One such option is what we did for c, where we have an invalid bit to tell whether more than 8 caches have the block. The bit can be used to trigger an invalidate broadcast when GetM is received in the block (all caches are potentially invalidated for this block).

Question 2

Op	Result	P1	P2	P3
-	-	I	I	I
r1	Full transfer (70)	S	I	I
r2	Full transfer (70)	S	S	I
r3	Full transfer (70)	S	S	S
w1	Write-through, write update (50)	S	S	S
w2	Write-through, write update (50)	S	S	S
w3	Write-through, write update (50)	S	S	S
r1	Cache hit (1)	S	S	S
r2	Cache hit (1)	S	S	S
r3	Cache hit (1)	S	S	S

Total: 363 cycles

Op	Result	P1	P2	P3	P4
-	-	I	I	I	I
r1	Full transfer (70)	S	I	I	I

w1	Write-through, write update (50)	S	I	I	I
r2	Full transfer (70)	S	S	I	I
w2	Write-through, write update (50)	S	S	I	I
r3	Full transfer (70)	S	S	S	I
w3	Write-through, write update (50)	S	S	S	I
r4	Full transfer (70)	S	S	S	S
w4	Write-through, write update (50)	S	S	S	S

Total: 480 cycles

Question 3

Assume - = Yes (Abort), No (Intervene).

MSI

Instruction	P1	P2	P3	Rd mem	Wr mem
R3	I	I	S	Y (Y)	N (N)
W3	I	I	M	N (Y)	N (N)
R2	I	S	S	- -	Y (Y)
W2	I	M	I	N (Y)	N (N)
W3	I	I	M	- -	Y (N)
W1	M	I	I	- -	Y (N)
R2	S	S	I	- -	Y (Y)

MESI

Instruction	P1	P2	P3	Rd mem	Wr mem
R3	I	I	E	Y (Y)	N (N)
W3	I	I	M	N (N)	N (N)
R2	I	S	S	- -	Y (Y)
W2	I	M	I	N (Y)	N (N)
W3	I	I	M	- -	Y (N)
W1	M	I	I	- -	Y (N)
R2	S	S	I	- -	Y (Y)

MOESI

Instruction	P1	P2	P3	Rd mem	Wr mem
R3	I	I	E	Y (Y)	N (N)
W3	I	I	M	N (N)	N (N)
R2	I	S	O	N (N)	N (N)
W2	I	M	I	N (N)	Y (N)
W3	I	I	M	- -	Y (N)
W1	M	I	I	- -	Y (N)

R2 O S I N - N (N)

Question 4

```
LL, SC *assuming x is the address
FetchAndAdd(addr x, int d) {
do {
        R2 = d;
        LL R1, x // load link from address x
        // set link register to x
        ADD R0, R1, R2 // R0 = R1 + R2 = mem[x] + d
        // if link register still set to x
        SC R0, x // store the result back into addr x
        // R0 gets overwritten to equal 1, if successful
        // else R0 == 0
        } while(R0 == 0); // don't need to test R1
}
```

Because the link register is internal to each processor, a snoop-based protocol will work much better with LL, SC. The cache controller will be able to see if one processor is invalidating the memory address stored in the link register. In the snoop-protocol, the bus lets others snoop on this invalidation, and each cache will be able to communicate to its processor quickly. However, the global directory-based protocol has no bus for snooping on an invalidation, so in a point-to-point network, these extra messages will be needed in order for the many processors to communicate with each other about the link register.